

# Samadhan Setu — Full Frontend Technical Manual

Complete Architecture, Component Workflows, Mapping Engine & 10 Presentation Q&As;

## 1. Executive Summary & System Architecture

Samadhan Setu is a crowdsourcing & collaboration platform connecting citizens reporting localized grassroots challenges with academic institutions, industry CSR cells, and government administrators. The frontend is engineered as a modern Single Page Application (SPA) prioritizing visual responsiveness, interactive GIS mapping, real-time client-side validation, and role-based interface adaptation.

| Component Layer       | Technology & Package                           | Technical Function & Value Proposition                                                                                                                |
|-----------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core SPA Framework    | React 18.3 + Vite 5.2                          | Fast HMR bundling, declarative component rendering & strict state isolation.                                                                          |
| Client Routing        | React Router DOM 6.23                          | Client-side routing with dynamic parameter matching ( <code>/challenges/:id</code> ) & <code>ProtectedRoute</code> wrappers.                          |
| UI Styling & Icons    | Tailwind CSS 3.4 + Lucide React                | Utility-first styling system, responsive grid layouts, and vector icon primitives.                                                                    |
| Geospatial GIS Engine | Leaflet 1.9 + React-Leaflet 4.2                | OpenStreetMap interactive coordinate picker, GPS geolocation & color-coded severity pins.                                                             |
| HTTP Client & Auth    | Axios 1.7 + Interceptors                       | Centralized API client with automated JWT bearer token injection via HTTP headers.                                                                    |
| Global Session State  | React Context API ( <code>AuthContext</code> ) | Persists authentication session, token lifecycle, and role evaluation ( <code>'citizen'</code> , <code>'institution'</code> , <code>'admin'</code> ). |

## 2. Frontend Directory Structure & File Map

- `src/main.jsx`: Application root entry point. Mounts `App.jsx` into the DOM `div#root` inside standard React `StrictMode`.
- `src/App.jsx`: Root routing configuration containing top-level Navbar, footer layout, and route switch definitions (`/`, `/login`, `/register`, `/create-challenge`, `/challenges/:id`, `/my-challenges`, `/admin`, `/institution`).
- `src/context/AuthContext.jsx`: Provides `AuthContext` provider. Stores user profile and JWT token in `localStorage`, performs `/api/auth/me` validation on app launch, and exposes `login`, `register`, and `logout` methods.
- `src/services/api.js`: Axios instance pre-configured with `baseUrl: '/api'`. Attaches `Authorization: Bearer` header to all outgoing requests if token exists in `localStorage`.
- `src/components/ProtectedRoute.jsx`: Route guard wrapper preventing unauthorized access. Redirects unauthenticated users to `/login` and role-mismatched users to `/`.
- `src/components/LocationPickerMap.jsx`: Interactive GIS map picker. Handles click events to capture exact `lat`/`lng` coordinates, marker dragging (`dragend`), HTML5 Geolocation (`navigator.geolocation`), and map recentering.
- `src/components/ChallengeMapDisplay.jsx`: Dual-mode Leaflet display component. Renders single challenge marker with Google Maps directions link OR multi-marker public feed map with severity-coded SVG icons.
- `src/components/StatusTimeline.jsx`: Renders the 5-stage lifecycle stepper (`Pending` → `Open` → `Under Review` → `In Progress` → `Resolved`) with status notes and historical audit timestamps.
- `src/components/ChallengeCard.jsx`: Card UI item for public feeds showing title, category, location, severity badge, affected count, vote count, and status badge.

### 3. Detailed View & Page Breakdown

#### Public Feed (`Home.jsx`)

Serves as the main discovery hub. Features a full-text search input (filtering title, location, description), status category filters, and a **\*\*Grid / Map View toggle switch\*\***. Uses `ChallengeCard.jsx` for grid view and `ChallengeMapDisplay.jsx` for multi-pin interactive GIS map view.

#### Challenge Intake (`CreateChallenge.jsx`)

Citizen report creation form. Requires exact state, district, locality, landmark, pincode, severity level, affected count, and who is affected. Features a 400ms debounced auto-search against `/api/challenges/similar` to surface existing duplicate reports, encouraging citizens to upvote existing issues. Embeds `LocationPickerMap.jsx` and transmits multi-part binary media uploads via `FormData`.

#### Challenge Detail (`ChallengeDetail.jsx`)

Comprehensive single-issue page. Displays ground evidence photo gallery, `ChallengeMapDisplay.jsx`, 5-stage `StatusTimeline.jsx`, Admin control actions (Approve, Mark Resolved), Institutional proposal submission form, assigned proposal details, execution progress evidence timeline, and community comment thread.

#### Admin Portal (`AdminDashboard.jsx`)

Moderation workspace. Lists pending challenges awaiting verification, displays duplicate score metrics, enables report merging, and manages user privileges.

#### Institution Portal (`InstitutionDashboard.jsx`)

Partner workspace for academic and CSR bodies to filter open challenges by domain, track proposal evaluation statuses, and upload progress evidence.

### 4. Step-by-Step Presentation Script for Presenters

**Step 1: Introduction (Slide 1)** — 'Good morning/afternoon evaluators. I am representing the Frontend Development team for Samadhan Setu. Our frontend is a responsive, accessible React 18 SPA built with Vite and Tailwind CSS designed to empower citizens to report grassroots issues with exact geospatial precision.'

**Step 2: GIS Map Integration Demo (Slide 2)** — 'A major innovation in our platform is moving beyond text-only locations. We built `LocationPickerMap` using Leaflet and OpenStreetMap. Citizens can click on the map or tap "Use My Current Location" via HTML5 Geolocation to capture exact latitude and longitude coordinates without relying on expensive proprietary APIs.'

**Step 3: Duplicate Mitigation & UX (Slide 3)** — 'To prevent platform clutter, as a user types a challenge title, our frontend debounces input by 400ms and queries the backend duplicate engine. If similar issues exist nearby, an amber card alerts the user to upvote the existing report instead of creating a duplicate.'

**Step 4: Lifecycle Stepper & Role Adaptation (Slide 4)** — 'Our detail page dynamically adapts based on user role (`AuthContext`). Citizens see upvoting and community discussions; accredited institutions see proposal submission forms; admins see status controls and report merging tools, all linked to a 5-stage visual `StatusTimeline` stepper.'

### 5. Top 10 Technical Presentation Q&As; (Frontend)

#### Q1: How is authentication managed and persisted across page refreshes?

**Answer:** Auth state is managed via `AuthContext.jsx`. Upon login or registration, the backend returns a JSON Web Token (JWT) and user profile object. The token is stored in `localStorage`. On application mount, an `useEffect` in `AuthContext` checks `localStorage` and queries `/api/auth/me` to revalidate and restore state. Axios interceptors automatically attach `Authorization: Bearer ` to all HTTP requests.

**Q2: How does the interactive map integration work without paid Google Maps API keys?**

**Answer:** We integrated Leaflet JS (`leaflet` & `react-leaflet``) with free OpenStreetMap tile servers (`https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`). `LocationPickerMap`` captures map click events to set precise `latitude`` and `longitude`` coordinates, while also offering an HTML5 Geolocation (`navigator.geolocation``) button. We created custom dynamic SVG marker pins color-coded by severity (Red for Critical, Orange for High, Amber for Medium) rendered via Leaflet's `L.divIcon``.

**Q3: How is real-time duplicate challenge prevention handled on the frontend?**

**Answer:** As the citizen types a title in `CreateChallenge.jsx``, a `useEffect`` trigger with a 400ms debounce timer sends a query to `/api/challenges/similar?title=...&category;=...`. If similar existing issues are found in that district/locality, an alert card instantly displays matching reports with direct links, prompting the user to upvote existing issues rather than creating duplicates.

**Q4: How are file uploads (ground photos) transmitted from React to the backend?**

**Answer:** Since image files cannot be sent as standard JSON, we use browser native `FormData``. In `CreateChallenge.jsx`` and `ChallengeDetail.jsx``, image files selected via ```` are appended to a `FormData`` instance along with text fields, and sent via Axios with `"Content-Type": "multipart/form-data"`. This enables binary file streaming to Express `"multer"`` middleware.

**Q5: How is Role-Based Access Control (RBAC) enforced on the UI?**

**Answer:** RBAC is enforced both visually and structurally. `ProtectedRoute.jsx`` wraps private routes, redirecting unauthenticated users to `/login`` or unauthorized roles to `/``. Within pages like `ChallengeDetail.jsx``, conditional rendering inspects `user.role``: citizens see the Upvote button; institutions see the Proposal Submission form; admins see approval and status transition buttons.

**Q6: Why did you choose Vite over Create React App (CRA)?**

**Answer:** Vite utilizes native ES modules (ESM) and esbuild pre-bundling during development, providing instant server start times and sub-millisecond Hot Module Replacement (HMR) regardless of application size. In production, Vite uses Rollup for highly optimized chunking and tree-shaking, resulting in smaller bundle sizes (e.g. 466 kB JavaScript bundle built in ~2 seconds).

**Q7: How do you handle map recentering when coordinates change externally (e.g. via Geolocation)?**

**Answer:** In `LocationPickerMap.jsx``, we created a helper child component `RecenterMap`` that consumes the `useMap()`` hook provided by `react-leaflet``. When `position`` state updates via browser geolocation, `RecenterMap`` executes `map.setView([position.lat, position.lng], 15)`` to smoothly animate and re-center the Leaflet map view on the user's GPS coordinates.

**Q8: How is state synchronization maintained between Grid View and Map View on the Home page?**

**Answer:** `Home.jsx`` maintains a single master state array `challenges`` fetched from `/api/challenges`` based on active search queries and filters. The `viewMode`` toggle state (`"grid" | "map"`) simply determines whether `filteredChallenges`` array is passed to `ChallengeCard`` grid items or the `ChallengeMapDisplay`` multi-marker Leaflet component. Filtering one automatically updates both views synchronously.

**Q9: How are Leaflet default marker URL missing issues resolved in modern JavaScript bundlers?**

**Answer:** Modern bundlers like Vite/Webpack rewrite asset paths, breaking Leaflet's default marker image relative URLs. We solved this by explicitly resetting `L.Icon.Default.prototype._getIconUrl`` and overriding default options with static CDN URLs (`unpkg.com/leaflet@1.9.4/dist/images/marker-icon.png``), ensuring markers render reliably in all build environments.

**Q10: What measures are taken to optimize accessibility and user experience across devices?**

**Answer:** Tailwind CSS standardizes responsive mobile-first layouts using breakpoint utilities (`"sm:", "md:", "lg:"`). Form inputs include clear label associations, ARIA attributes, semantic HTML tags (`"`, ``", ``"`), contrast-compliant color scales (Slate-900, Brand-600), and fallback loading spinners during asynchronous network requests.